

Vite & Gourmand

Traiteur professionnel — Bordeaux & Gironde

Documentation technique

Version 1.0 — Mai 2026

Projet ECF DWWM — Studi

Auteur : Loïc Rovelas

Sommaire

1. Réflexions initiales technologiques	p. 2
2. Configuration de l'environnement de travail	p. 3
3. Modèle conceptuel de données (MCD)	p. 4
4. Diagramme des cas d'utilisation	p. 6
5. Diagrammes de séquence	p. 7
6. Sécurité	p. 9
7. Documentation du déploiement	p. 10

1. Réflexions initiales technologiques

Avant de démarrer le développement, une analyse comparative des technologies disponibles a été réalisée afin de choisir la stack la plus adaptée aux contraintes du projet : hébergement mutualisé ou cloud, compétences disponibles, maturité des outils.

1.1 Back-end

PHP 8.3 + PDO

PHP est le langage back-end le plus répandu en hébergement mutualisé et correspond à la stack de l'ECF. PDO offre une couche d'abstraction sécurisée (requêtes préparées) indépendante du SGBD. Alternatives envisagées : Node.js (Express) et Python (Django). PHP a été retenu pour sa compatibilité avec Laragon en développement local et sa simplicité de déploiement.

1.2 Base de données relationnelle

MySQL 8

MySQL est intégré nativement dans Laragon (environnement local) et proposé comme plugin dans Railway (production). Sa maturité, sa documentation abondante et sa parfaite compatibilité avec PDO en font le choix naturel. PostgreSQL aurait été pertinent pour des fonctionnalités avancées (JSONB, triggers), non nécessaires ici.

1.3 Base de données NoSQL

MongoDB Atlas

MongoDB est choisi pour stocker les données analytiques des commandes (document JSON flexible) et alimenter le tableau de bord statistique de l'administrateur. L'utilisation de MongoDB Atlas (tier M0 gratuit) évite d'avoir à gérer une instance MongoDB en production. Chaque nouvelle commande est insérée dans les deux bases : MySQL pour les données opérationnelles, MongoDB pour les statistiques.

1.4 Front-end

Bootstrap 5 + JS

Bootstrap 5 permet un développement rapide d'une interface responsive et accessible (RGAA). Il évite l'ajout d'un framework JavaScript lourd (React, Vue) non nécessaire pour une application majoritairement rendue côté serveur. Le JavaScript vanilla est utilisé pour les interactions dynamiques (filtres menus sans rechargement, calcul de prix en temps réel).

1.5 Serveur web

FrankenPHP

FrankenPHP est un serveur d'application PHP moderne basé sur Caddy, proposé par l'image Docker officielle [dunglas/frankenphp](https://github.com/dunglas/frankenphp). Il intègre PHP nativement (pas de PHP-FPM séparé), supporte HTTP/2 et la compression gzip. Choisi pour sa simplicité de configuration en production sur Railway et son image Docker officielle.

1.6 Hébergement

Railway

Railway offre un déploiement automatique depuis GitHub (push → build → deploy), un plugin MySQL intégré, le support Docker et des variables d'environnement gérées via interface. Alternatives : fly.io (complexité réseau), Heroku (payant), Vercel (orienté Node.js). Railway a été retenu pour sa facilité d'usage et son tier gratuit suffisant pour l'ECF.

1.7 Envoi d'e-mails

Brevo HTTP API

Railway bloque les connexions SMTP sortantes (port 587), rendant PHPMailer inutilisable. L'API HTTP de Brevo (port 443, toujours ouvert) a été adoptée comme alternative. Les appels se font avec curl directement depuis PHP, sans librairie tierce supplémentaire.

2. Configuration de l'environnement de travail

2.1 Environnement local

Outil	Version	Rôle
Laragon	6.x	Serveur local Windows (Apache, PHP, MySQL intégrés)
PHP	8.3	Langage back-end, extensions : pdo_mysql, mongodb, zip, curl
MySQL	8.0	Base de données relationnelle locale
Composer	2.x	Gestionnaire de dépendances PHP
Git	2.x	Gestion de version
Visual Studio Code	Latest	Éditeur de code principal
phpMyAdmin	Inclus Laragon	Administration base de données locale

2.2 Environnement de production

Outil	Rôle
Railway	Plateforme de déploiement (PaaS), héberge le container Docker et la base MySQL
Docker / FrankenPHP	Serveur web PHP en production (<code>dunglas/frankenphp:latest-php8.3</code>)
GitHub	Dépôt de code source, déclencheur de déploiement (push → Railway build)
MongoDB Atlas	Base NoSQL hébergée (tier M0 gratuit), statistiques commandes
Brevo	Service d'envoi d'e-mails transactionnels (API HTTP)

2.3 Dépendances PHP (Composer)

Package	Version	Utilisation
<code>mongodb/mongodb</code>	^2.0	Client PHP pour MongoDB Atlas (requêtes, agrégations)
<code>phpmailer/phpmailer</code>	^6.9	Présent comme dépendance, remplacé en prod par l'API Brevo directe

2.4 Structure du projet

Dossier / Fichier	Contenu
<code>assets/</code>	Images, logo SVG, icônes
<code>CSS/</code>	Feuilles de style (style.css, espace-employe.css, espace-admin.css...)
<code>HTML/</code>	Pages PHP publiques + espaces connectés (admin, employé, utilisateur)

Dossier / Fichier	Contenu
<code>JS/</code>	Scripts JavaScript (filtres menus, calcul prix commande)
<code>PHP/config/</code>	Connexions BDD : <code>db.php</code> (MySQL/PDO), <code>mongodb.php</code>
<code>PHP/includes/</code>	Fonctions transversales : <code>session.php</code> , <code>mailer.php</code>
<code>SQL/</code>	<code>create.sql</code> (DDL), <code>seed.sql</code> (données de démonstration)
<code>Dockerfile</code>	Image Docker production (FrankenPHP + extensions PHP)
<code>Caddyfile</code>	Configuration du serveur Caddy (redirections, headers sécurité)
<code>php-session.ini</code>	Configuration PHP sessions injectée dans le container Docker

3. Modèle conceptuel de données (MCD)

Le schéma ci-dessous correspond à l'annexe 1 du sujet ECF, complété et implémenté dans

SQL/create.sql .

3.1 Entités principales

UTILISATEUR PK utilisateur_id email (UNIQUE) password (bcrypt) nom, prenom telephone adresse, code_postal ville, pays FK role_id actif (BOOL) token_reset	MENU PK menu_id titre description nb_personne_minimum prix_par_personne conditions quantite_restante FK theme_id FK regime_id actif (BOOL)	COMMANDE PK commande_id numero_commande (UNIQUE) FK utilisateur_id FK menu_id date_commande date_prestation heure_livraison adresse_prestation nombre_personne prix_menu, prix_livraison prix_total statut (ENUM) pret_materiel (BOOL)
PLAT PK plat_id nom type (entrée/plat/dessert) photo (BLOB)	AVIS PK avis_id FK utilisateur_id FK commande_id note (1-5) commentaire statut (en_attente/valide/refuse)	
Tables de référence ROLE (role_id, libelle) THEME (theme_id, libelle) REGIME (regime_id, libelle) ALLERGENE (allergene_id, libelle) HORAIRE (jour, heure_ouverture...)	Tables d'association MENU_PLAT (menu_id, plat_id) PLAT_ALLERGENE (plat_id, allergene_id) COMMANDE_HISTORIQUE (commande_id, statut, date)	

3.2 Relations principales

UTILISATEUR	1,N	passe	1,1	COMMANDE
COMMANDE	N,1	porte sur	1,1	MENU
MENU	N,N	propose	N,N	PLAT
PLAT	N,N	contient	N,N	ALLERGENE
UTILISATEUR	1,N	dépose	1,1	AVIS
COMMANDE	1,N	a pour historique	1,1	COMMANDE_HISTORIQUE

UTILISATEUR	N,1	a un	1,1	ROLE
MENU	N,1	a un	1,1	THEME
MENU	N,1	a un	1,1	REGIME

3.3 Collection MongoDB (NoSQL)

En parallèle de MySQL, chaque commande validée est insérée dans la collection `commandes` de la base MongoDB `vite_gourmand` :

Champ	Type	Description
<code>commande_id</code>	Int	Référence vers la commande MySQL
<code>menu_id</code>	Int	Identifiant du menu commandé
<code>menu_titre</code>	String	Titre dénormalisé du menu
<code>prix_total</code>	Double	Montant total de la commande
<code>nombre_personne</code>	Int	Nombre de convives
<code>date_commande</code>	UTCDateTime	Horodatage de la commande

4. Diagramme des cas d'utilisation

Les cas d'utilisation sont organisés par acteur. Chaque acteur hérite des droits de l'acteur inférieur.

Visiteur (non authentifié)

- Consulter la page d'accueil et les avis validés
- Consulter la liste des menus avec filtres dynamiques
- Consulter le détail d'un menu (plats, allergènes, conditions, stock)
- Créer un compte utilisateur
- Se connecter
- Réinitialiser son mot de passe par e-mail
- Contacter l'entreprise via le formulaire
- Consulter les mentions légales et les CGV

Utilisateur (authentifié, rôle 3)

- *Hérite de tous les cas du visiteur*
- Commander un menu (avec calcul du prix, réduction et frais de livraison)
- Modifier une commande en attente
- Annuler une commande en attente
- Suivre l'avancement de sa commande (historique des statuts)
- Déposer un avis (note + commentaire) sur une commande terminée
- Modifier ses informations personnelles

Employé (authentifié, rôle 2)

- *Hérite de tous les cas de l'utilisateur*
- Créer, modifier et supprimer des menus
- Créer, modifier et supprimer des plats et leurs allergènes
- Modifier les horaires d'ouverture
- Consulter et filtrer les commandes (par statut, par client)
- Faire avancer le statut d'une commande
- Annuler une commande (avec motif et mode de contact)
- Valider ou refuser un avis client

Administrateur (authentifié, rôle 1)

- *Hérite de tous les cas de l'employé*
- Créer un compte employé
- Activer / désactiver un compte employé
- Consulter le tableau de bord statistique (MongoDB) : commandes par menu, CA par menu

- Filtrer les statistiques par menu et par période

5. Diagrammes de séquence

5.1 Connexion utilisateur

Utilisateur		Navigateur	PHP (connexion.php)		MySQL
Saisit email + mot de passe	→	POST /HTML/connexion.php			
			Vérifie token CSRF	→	SELECT utilisateur WHERE email = ?
			password_verify() + actif = 1	←	Retourne l'enregistrement
			\$_SESSION['utilisateur_id'] = ...		
Redirigé vers Mon espace	←	Location: /HTML/espace-*/index.php			

5.2 Passer une commande

Utilisateur		commande.php		MySQL		MongoDB / Brevo
Clique "Commander" sur un menu	→	GET commande.php? menu_id=X	→	SELECT menu WHERE menu_id = X		
Remplit et soumet le formulaire	→	POST commande.php				
		Vérifie CSRF + stock + minimum	→	BEGIN TRANSACTION		
		Calcule prix + livraison + réduction	→	INSERT commande + historique		
			→	UPDATE quantite_restante - 1		
			→	COMMIT	→	insertOne() dans MongoDB
					→	sendMail() confirmation Brevo
Page confirmation	←	Redirect commande-confirmation.php				

5.3 Changement de statut d'une commande (Employé)

Employé		espace- employe/index.php		MySQL		Brevo
Clique sur nouveau statut	→	POST action=changer_statut				
		Vérifie CSRF + transition valide	→	UPDATE commande SET statut = ?		
			→	INSERT commande_historique		
		Si statut = "attente_retour_materiel"			→	mailRetourMateriel()
		Si statut = "terminee"			→	mailCommandeTerminee()
Page rechargée avec nouveau statut	←	Redirect				

6. Sécurité

Les mesures de sécurité suivantes ont été mises en place, conformément aux exigences RGPD et OWASP :

- CSRF** Tous les formulaires POST sont protégés par un token CSRF généré via `bin2hex(random_bytes(32))` , stocké en session et vérifié avec `hash_equals()` avant tout traitement.
- Mots de passe** Les mots de passe sont hachés avec `password_hash()` (bcrypt, coût 10) et vérifiés avec `password_verify()` . Politique de mot de passe : 10 caractères minimum, majuscule, minuscule, chiffre, caractère spécial.
- Injections SQL** Toutes les requêtes utilisent PDO avec des requêtes préparées et des paramètres liés. Aucune concaténation de données utilisateur dans les requêtes SQL.
- XSS** Toutes les données affichées en HTML sont échappées via `htmlspecialchars()` . Les headers HTTP `X-Content-Type-Options` et `X-Frame-Options: DENY` sont configurés dans le Caddyfile.
- Sessions** Sessions configurées avec `cookie_httponly=1` , `cookie_secure=1` , `samesite=Lax` et `use_strict_mode=1` . Le token CSRF est régénéré après chaque soumission réussie.
- Contrôle d'accès** Chaque page protégée appelle `requireConnexion()` ou `requireRole()` en début de fichier. Redirection immédiate si l'utilisateur n'a pas le rôle requis.
- Reset password** Le lien de réinitialisation contient un token aléatoire (`random_bytes(32)`) stocké hashé en base, avec une expiration de 1 heure. Le token est supprimé après utilisation.
- Uploads** Les images des plats sont stockées en base (BLOB) après vérification du type MIME via `finfo_file()` . Seuls les types image/jpeg, image/png et image/webp sont acceptés.
- RGPD** Les données personnelles (adresse, téléphone) ne sont accessibles qu'aux utilisateurs concernés et aux employés/admin. Page mentions légales et CGV disponibles sans authentification.

7. Documentation du déploiement

7.1 Architecture de production

Composant	Service	Détail
Application PHP	Railway (Docker)	Image FrankenPHP, build depuis Dockerfile
Base MySQL	Railway Plugin MySQL	Variables DB_HOST, DB_PORT, DB_NAME injectées automatiquement
Base MongoDB	MongoDB Atlas (M0)	Cluster cloud gratuit, connexion via MONGODB_URI
E-mails	Brevo API HTTP	Clé API stockée dans BREVO_API_KEY, port 443
DNS / HTTPS	Railway	Certificat TLS automatique (Let's Encrypt via Caddy)

7.2 Dockerfile

L'image de production est basée sur `dunglas/frankenphp:latest-php8.3` :

1. Installation des extensions PHP : `pdo_mysql` , `mongodb` , `zip`
2. Injection du fichier `php-session.ini` (`output_buffering`, sécurité des cookies)
3. Installation des dépendances Composer en mode production (`--no-dev --optimize-autoloader`)
4. Copie des fichiers du projet
5. Remplacement du Caddyfile par défaut par le Caddyfile du projet

Point critique : La directive `output_buffering = 4096` dans `php-session.ini` est indispensable. FrankenPHP marque les headers HTTP comme "déjà envoyés" avant l'exécution du code PHP, ce qui empêche `session_start()` de fonctionner. Ce paramètre, présent par défaut dans Laragon, doit être explicitement configuré dans le container Docker.

7.3 Étapes de déploiement initial

- 1 Créer le service Railway**
Nouveau projet → Deploy from GitHub → sélectionner le dépôt `Neyth97/vite-et-gourmand` .
- 2 Ajouter le plugin MySQL**
Dans Railway → + New → Database → MySQL. Les variables `DB_*` sont injectées automatiquement dans le service.
- 3 Configurer les variables d'environnement**
Dans le service → Variables : ajouter `MONGODB_URI` , `APP_URL` , `BREVO_API_KEY` , `MAIL_FROM` , `MAIL_FROM_NAME` .
- 4 Initialiser la base MySQL**
Via Railway → MySQL → Connect → Query : exécuter `SQL/create.sql` puis `SQL/seed.sql` .
- 5 Configurer MongoDB Atlas**

Créer un cluster M0 → Database Access (créer un utilisateur) → Network Access (autoriser `0.0.0.0/0` pour les IPs dynamiques de Railway) → récupérer l'URI de connexion.

6

Déploiement continu

Tout push sur la branche `main` déclenche automatiquement un nouveau build et déploiement sur Railway. Durée moyenne du build : 2 à 3 minutes.

7.4 Variables d'environnement requises

Variable	Obligatoire	Description
<code>DB_HOST</code>	Oui (auto)	Injectée par le plugin Railway MySQL
<code>DB_PORT</code>	Oui (auto)	Injectée par le plugin Railway MySQL
<code>DB_NAME</code>	Oui (auto)	Injectée par le plugin Railway MySQL
<code>DB_USER</code>	Oui (auto)	Injectée par le plugin Railway MySQL
<code>DB_PASS</code>	Oui (auto)	Injectée par le plugin Railway MySQL
<code>MONGODB_URI</code>	Oui	URI de connexion MongoDB Atlas
<code>APP_URL</code>	Oui	URL publique de l'application (pour les liens dans les e-mails)
<code>BREVO_API_KEY</code>	Oui	Clé API Brevo (commence par <code>xkeysib-</code>)
<code>MAIL_FROM</code>	Oui	Adresse expéditeur vérifiée dans Brevo
<code>MAIL_FROM_NAME</code>	Non	Nom affiché de l'expéditeur (défaut : Vite & Gourmand)
<code>APP_ENV</code>	Non	Mettre <code>production</code> pour désactiver les erreurs PHP